Direct Publication for Offline Ballots

An Executable Research Prototype

João Henrique Menezes Silva Boia

Independent researcher

October 2026

Abstract

This paper asks which locally valid copy of a signed batch can still be published after one fault, and which board rejections leave a later authentic post possible. Three policies read the same machine: memory only, the transported file only, or either source. They share one local signature check, the same two attempts, and the same stops. Invalid bytes are not sent. Nothing is sent without the machine key, a connection, a reachable board, and an open deadline, and nothing is sent again after this machine has published. On one local board, the policy that may read either source publishes the intact file when memory was altered, and publishes memory when the file was altered or is absent. A policy that cannot read the surviving copy publishes nothing. That is availability of a valid source, not a different acceptance rule. A separate run skips the local check and mines the reverting transaction. `WrongSigner`, `UnregisteredMachine`, and `BatchWindowClosed` do not set the published flag, so the intact batch can still be posted while the deadline is open. After one accepted post, a second post reverts with `BatchAlreadyPosted` and leaves the first batch in place. Every included post in this run used 339,720 gas. The mined reverting receipts returned the same `gasUsed`. The call trace did not match that receipt: the early reverts made no signature-precompile call, while `WrongSigner` and the inclusion did. The receipt is not used as a revert-cost comparison. The comparison does not estimate how often a fault occurs. A hand-marked paper ballot remains the stronger record.

1. Introduction

A polling-place count often gives the operator two jobs. The machine, or the official who reads its media, announces the station total. And the complete record leaves on a memory card or through a person who is supposed to forward it. This prototype takes the announced total off the machine: the tally is recomputed from the published encrypted ballots and from three of five trustee openings of their sum. The question measured here is the second job.

When does the machine submitting its own signed batch change the audit outcome, relative to forwarding that same batch? A signature does not answer it. The same bytes verify the same way however they arrive. What changes is whether a valid copy is still available to the only account allowed to post it. Someone who holds the only file can withhold it, or the file can be lost or damaged. For a policy that can read only that file, the audit sees one fact: a registered machine has no batch. The omission is that missing machine. Another account cannot post in its place.

Section 4.2 runs three policies from the same unpublished snapshot. They share the local check, the two slots, and the rule that a published machine is not sent again. They differ only in which sources they read.

When memory is altered and the file is intact, the either-source policy does not send the altered memory; it sends the file. A mined rejection is a different experiment. It does not mark the machine published, so a later intact post is still accepted while the deadline remains open. An included transaction of one payload has one receipt gas on every policy. The Anvil receipt for a mined revert used that same number, so this paper does not report a separate revert cost. The ballot constructions are exponential ElGamal, a Chaum–Pedersen proof, and Shamir sharing, used as published. A hand-marked paper ballot remains the stronger record, because the voter marks it and an audit can correct the total. The other limits are collected in Section 7.

2. Implementation and evidence

2.1 Roles

Table 1 is that split. The Brazilian electoral authority documents online publication of polling-station bulletins, digital vote records, and logs, a QR-code format that third parties may implement, and procedures for source-code inspection [11]. The bulletin is a station total. The digital records leave the machine on media, and the authority's own rules cover recovery from that media or from the printed bulletin [12]. A third party can read what was published. That is not the machine submitting the ballots itself, and it is not an individual ciphertext receipt. The citations are the documented procedures, not a claim that those procedures were abused.

Table 1: Implemented interfaces and the evidence each one handles.

Step	Operation	Evidence or check
Ballots	Machine encrypts the selection	Bit and exactly-one proofs
Receipt	Issued after selection, before CONFIRM or TEST	Signed ciphertext commitment
Publication	Machine submits its finalized batch	Domain-bound batch signature
Board	Records one batch per machine	Registry, signature, deadline
Opening	Trustees submit aggregate contributions	Aggregate-bound decryption proofs
Verification	Separate program processes public files	Batch completeness, receipts, proofs, tally

If the same signed bytes arrive, the signature check is the same whether a person forwarded them or the machine submitted them. Section 4.2 asks which attempt is still allowed after a fault. It does not change that check. The carrier, when it is written, is a copy, and holding it does not supply the machine credential. The early-network check refuses to enable transport before the close. A receipt holder can check that the committed ciphertext is inside a published batch.

The dealer is trusted, the demonstration process holds all five shares, and any three colluding trustees can decrypt individual ciphertexts; the aggregate-only interface is not a cryptographic barrier to collusion. The verifier requires complete authentic evidence. A malicious machine can encrypt a choice different from the voter's intent. Since the receipt follows selection, detecting this requires an independent voter comparison with the intended candidate; the TEST consistency check alone does not establish intent. One local chain operator can censor or reset the chain, and copied event data is not proof of independent consensus.

2.2 Threat model and claim boundaries

The assets are choices and randomness before disclosure, key shares, batch integrity, retained receipts, complete board evidence, and the tally. Considered adversaries may control a machine, alter or omit transport data, submit rogue or duplicate batches, announce false totals, supply malformed proofs, censor evidence,

collude as trustees, coerce voters, or control the local chain and host. The prototype has no voter roll, eligibility checks, turnout reconciliation, or one-person-one-vote enforcement.

Assumptions. The manifest, trustee registry, public key, and exported board evidence must be authentic and complete. A trusted dealer creates 3-of-5 shares; there is no distributed key generation. Machine and host are not remotely attested. The local chain is one process, not independent consensus. TEST selection must be unpredictable to a malicious machine for challenge value, and voters must be able to compare the candidate independently. The integration examples do not establish these assumptions.

Properties checked. The contract checks registration, signatures, deadlines, and append rules. The off-chain verifier checks ballot proofs and openings, receipt signatures and inclusion, supplied partial proofs, and tally consistency. A signature catches an edit on either path. It does not catch a batch that was never submitted. The machine account submitting against the public machine list is what makes that omission visible. Eligibility, coercion resistance, and a board that stays available are not among these checks.

Collusion and post-detection limits. Any three trustees can decrypt arbitrary ciphertexts despite the aggregate-only workflow. The dealer is trusted, the demo process holds all shares, and the chain operator can censor or reset the local chain. The contract does not verify ballot or trustee proofs. Missing machine batches, invalid ballots, invalid supplied receipts, or inconsistent event hashes stop the audit without totals; the verifier does not discard ballots to produce a partial election result. Invalid trustee contributions are handled separately as described below. There is no automatic election annulment or dispute-resolution procedure. Media loss, withholding, access costs, exception handling, and preservation are operational scenarios, not findings about Brazil. The cited Brazilian rules describe integrity checks, designated technical recovery, recovery from available machine or media records or a printed BU, reasoned exception records, and preservation [12].

3. Protocol

The group is RFC 3526 group 14: a 2048-bit prime p, order $q = (p-1)/2$, and generator $g = 2$ [7]. Secret exponents lie in $\mathbb{Z}_q$, and the public key is $y = g^x$. For a bit m and fresh randomness r, exponential ElGamal is

$$\mathrm{Enc}_y(m; r) = (a, b) = (g^r,\ y^r g^m) \pmod{p}. \tag{1}$$

A ballot is one ciphertext per candidate. Each ciphertext carries a Fiat–Shamir OR proof that it encrypts zero or one. The challenge binds a domain separator, the election identifier, the public key, the whole ordered ciphertext vector, and the candidate index, so a proof cannot be moved to another election, key, ballot, or position. A second proof shows that the candidate bits sum to one. With $A = \prod_i a_i$, $B = \prod_i b_i$, and witness $R = \sum_i r_i \bmod q$,

$$\log_g A = \log_y (B/g) = R \pmod{q}. \tag{2}$$

The transcript is canonical JSON, hashed with SHA-256, and reduced modulo q. Subgroup membership, scalar ranges, encodings, and the proof equations are checked before a ballot is accepted. The equality proof follows Chaum and Pedersen [5]. The sharing follows Shamir [10]. Neither construction is new here.

3.1 Receipt, then confirm or test

The machine encrypts the selected choice and signs the receipt after selection but before the voter confirms or tests. The receipt binds the election, the chain, the board, the machine account, and a SHA-256 commitment to the ordered ciphertext vector. Proof bytes are not part of that commitment. A 20-hex-character code is

an 80-bit locator for lookup. It is not the binding value. Verification uses the full digest and the signature. Because the receipt follows selection, it is not independent evidence of what the voter intended.

Before emitting a receipt, the machine checks that either decision would fit the canonical batch-size limit, including the larger TEST opening. A capacity refusal emits no receipt and leaves previously accepted ballots unchanged. This does not provide crash persistence of the machine's in-memory state.

On TEST, the machine reveals the choice and randomness and marks the ballot spoiled. The verifier recomputes the ciphertexts and checks opening consistency. The revised machine-attack command exercises two distinct cases: an inconsistent opening fails cryptographically; a valid opening of a substituted choice passes cryptographic verification but fails a separate comparison with remembered voter intent. Disabling that comparison makes the second demonstration fail to detect the substitution. This is a simulated trusted comparison, not a validated human interface. Tested ballots do not count, and the voter votes again. On confirmation, randomness stays undisclosed. The receipt does not prove a physical button press or eligibility.

3.2 Direct publication

Local voting closes first. The voting program cannot enable transport before that close. After the close, the machine publishes one signed batch before the upload cutoff. The cutoff is not the same event as the local close: machines do not have to finish closing before any machine may upload. When the batch deadline passes, the board closes and a snapshot is fixed. Trustee shares are accepted only after that snapshot and before a separate deadline.

The batch is signed by the same secp256k1 account that submits the transaction. The EIP-191 digest binds the chain, the board, the election, and the payload. The contract checks the signer, the deadline, and the one-batch rule, then emits the payload and its hash. The verifier checks the event and the signature. Nothing in this step decrypts a ballot.

The demonstration submits the finalized batch without first writing a transport file. An optional carrier copy is staged, flushed, and file-synced before atomic no-clobber publication in the same directory. A failed write leaves no completed-looking destination and can be retried. Failure to create the optional copy is reported and does not stop direct submission of the other machine's batch. This mechanism does not guarantee directory-entry durability after power loss, recover lost memory, or make all copies available. Possessing the carrier does not supply the machine-account credential required for submission.

3.3 Opening only the sum

The verifier multiplies the ciphertexts of cast ballots after the board has closed. Spoiled ballots stay out. For a product (A, B) and a trustee share s_i, trustee i posts $D_i = A^{s_i}$ and a Chaum–Pedersen proof that $\log_g VK_i = \log_A D_i$, where $VK_i = g^{s_i}$. Each proof binds the election, the aggregate digest, the trustee index, the candidate, the key, the ciphertext, and the commitments. For a set S of three trustee indices,

$$\lambda_i = \prod_{\substack{j \in S \\ j \neq i}} \frac{-j}{i-j} \pmod{q}, \qquad D = \prod_{i \in S} D_i^{\lambda_i}. \tag{3}$$

The plaintext group element is $B/D = g^T$. The verifier solves for T inside the declared tally bound. The result is the aggregate. It is not a cryptographic guarantee against three trustees who decide to decrypt some other ciphertext.

A registered trustee may submit once. The revised board pins a digest only after three distinct trustees have posted that digest; one or two early conflicting submissions do not establish the pin. This agreement is not a

proof of mathematical correctness. The verifier recomputes the aggregate from all audited ballots, requires the pin to match it, and checks each contribution. Invalid trustee payloads, aggregate bindings, or proofs are excluded and their indices appear in `rejected_trustees`. At least three distinct valid contributions are still required. Duplicate trustee events or corrupted event hashes stop the audit. A successful report with rejected trustees verifies the tally from the valid threshold, not the validity of every submitted contribution.

4. What the checks catch

The eight commands below are integration examples showing that corresponding checks fire in this implementation. They are not a security proof. The fail-closed ballot checks and threshold-contribution exclusions described above are program behavior, not an adjudication or automatic annulment procedure.

- **TEST opening and intent.** The command separately demonstrates rejection of an inconsistent opening and a valid substituted-choice opening detected only by a simulated independent intent comparison (`attack-machine`). Untested ballots and human comparison effectiveness are not covered.
- **Edited transport.** Changing the signed bytes invalidates the signature, and the contract rejects the submission (`attack-transport`).
- **Dropped ballot.** A retained receipt that is absent from the signed batch fails the check (`attack-drop-vote`).
- **False tally.** An announced total that disagrees with the recomputation is rejected (`attack-tally`). The verifier uses the public evidence, not a private list of expected choices.
- **Two selections.** A ballot with two selected candidates fails the exactly-one proof (`attack-double-vote`). The proof does not stop someone from submitting several valid ballots.
- **Second batch.** The contract rejects another batch from a registered machine while the upload window is still open (`attack-duplicate`).
- **Unknown machine.** The contract rejects a signer that was never registered (`attack-rogue-machine`). Registration itself is an external anchor.
- **Early network.** The voting program cannot enable the simulated transport before the close (`attack-early-network`). This is a property of the software state, not of a physical switch.

If each ballot is tested independently with probability p after the receipt exists, and k ballots were altered, the chance of catching at least one alteration is $1 - (1 - p)^k$. Figure 1 plots that expression. It is not a measured detection rate. It assumes the decision is unpredictable and independent of what the machine knows. It does not model coercion, correlated tests, or malware that adapts only to ballots it expects to remain untested.

4.1 Regression scenarios in the revised implementation

Revision `8df8e3a` also contains focused regressions for four concrete failure paths. An artificially reduced payload limit makes the capacity boundary inexpensive to exercise: no new receipt is emitted when either decision would exceed it. Injected carrier-write and publication failures leave no partial destination, preserve existing files, and permit retry. A real local-contract scenario submits two wrong digests first, followed by

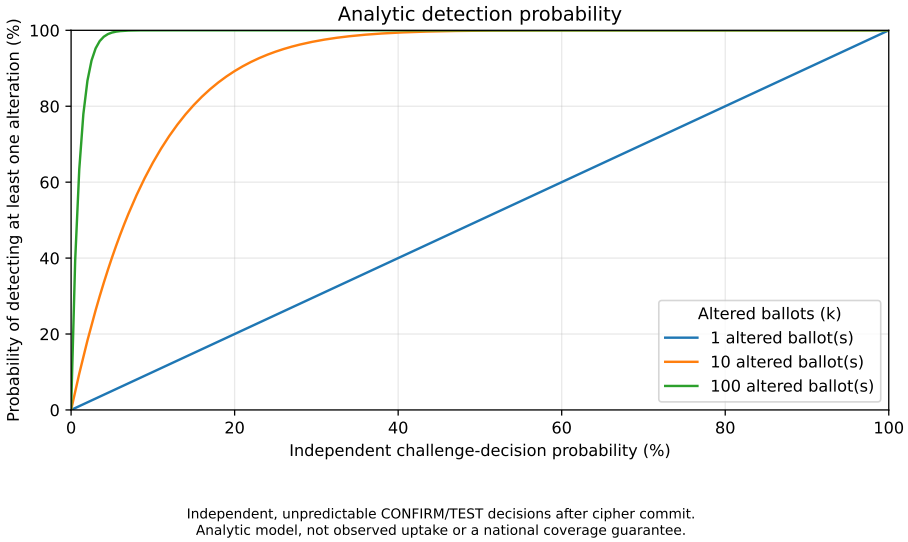


Figure 1: Analytic probability $1-(1-p)^k$ of detecting at least one altered ballot when tests are independent and chosen after the receipt exists. The curves are the equation, not observed detection rates.

three valid contributions; the verifier returns the expected tally and reports rejected trustees 1 and 2. A full synthetic election with both optional carrier writes disabled still publishes two batches and verifies totals $(2, 2, 2)$, with two explicit carrier warnings. These are bounded fault-injection observations, not measured attack probabilities, usability results, or evidence about another voting system.

4.2 Three source policies

The question is which valid source is still available after one fault, and which board rejection still leaves a later authentic post possible. All three policies start from the same signed batch and the same unpublished board. Each run is reverted to that snapshot before the next policy begins. The fault is applied before that policy chooses a source. Memory reads only the in-memory bytes. File reads only the transported file. Either reads memory when those bytes still match the machine signature, and otherwise reads the file. Each policy has the same two slots. A source is sent only when it is locally valid. No policy sends without the machine key, a connection, a reachable board, and an open deadline. No policy sends after this machine has published.

Those stops happen before a transaction. They are not a contract rejection. A contract rejection is a transaction the board reverts, and it is measured in a separate run that deliberately skips the local check. On this board the checks run in order. An unregistered account reverts with `UnregisteredMachine`. A late post reverts with `BatchWindowClosed`. A second post reverts with `BatchAlreadyPosted`. A signature that does not recover to the sender reverts with `WrongSigner`. The published flag is set only after those checks. Deleting the file after inclusion leaves the posted batch in place. Possessing the file does not supply the machine account.

Table 2 is the audit after ten faults. *In* means the closed board contained this batch and its one cast ballot. *Out* means the policy sent no transaction. An absent file is the same input whether someone withheld it or it was lost. The table is not a count of how often either act occurs. A link that fails only in the first slot is not distinguished by policy: all three send in the second slot.

In the comparison run reported here, every included transaction used 339,720 gas, whichever pol-

Table 2: Audit of one signed batch after one fault. Each cell is a separate run from the same unpublished snapshot. *In*: the batch and its one cast ballot were included. *Out*: no transaction was sent.

Fault before sending	Memory	File	Either
No fault	in	in	in
File absent before sending	in	out	in
Memory absent, file intact	out	in	in
Memory altered, file intact	out	in	in
File altered, memory intact	in	out	in
Both copies altered	out	out	out
Link down, then up	in	in	in
Link down, deadline closes before the later slot	out	out	out
Key absent	out	out	out
Board unreachable in both slots	out	out	out

icy selected the bytes. The receipt of each mined revert carried the same `gasUsed`: `WrongSigner`, `BatchAlreadyPosted`, `BatchWindowClosed`, and `UnregisteredMachine`. The call trace separates those transactions where the receipt does not. `UnregisteredMachine`, `BatchWindowClosed`, and `BatchAlreadyPosted` returned with no nested call. `WrongSigner` and the accepted post each made one call to the signature precompile. Because `gasUsed` did not move with that difference, this paper does not treat the receipt as a revert-cost measurement. Cells marked *out* recorded no gas, because no transaction was sent. The absolute figure follows the payload bytes. An earlier one-batch execution recorded 339,660, and another one-batch inclusion on Anvil 1.8.5 recorded 339,690. Those receipts are not combined with this run. These figures are not the baseline batches in Table 4.

The source difference is altered memory with an intact file. Memory has nothing it is allowed to send. File and either include the file, and either does so after refusing the altered memory locally. The mirror is an altered file with intact memory: file sends nothing, while memory and either include from memory. If both copies are altered, all three send nothing. A link that is down only in the first slot is not a contract rejection. All three include in the second slot. If the deadline passes before that slot, or the key is missing, or the board stays unreachable, all three stay out.

Skipping the local check shows the board's own boundary. A mined `WrongSigner` does not set the published flag, and the following intact post from the machine account is included. A mined `UnregisteredMachine` likewise leaves the machine unpublished, and the machine account can still post. After a successful post, a second transaction reverts with `BatchAlreadyPosted` and the audit still returns the first payload. After the deadline, the intact bytes revert with `BatchWindowClosed` and stay unpublished. The audit's missing-batch check is the set of registered machines that have no published batch. It does not depend on which of these policies submitted, and another account cannot fill the gap.

5. Measurements

The implementation has an offline Python machine, a Solidity contract, and a separate Python verifier. The reported run uses one local Anvil process. These observations describe the recorded baseline implementation, not necessarily revised code; they are not a public-chain fee schedule or a benchmark across hardware. Tool installation and the first container build are excluded. The recorded environment is arm, Python 3.12.11, Anvil 1.8.5, and Darwin 27.2.0.

The run contains six cast ballots, one spoiled ballot, three candidates, two machines, two batches, and three

Table 3: Pure-operation time in seconds from three samples, rounded to three decimal places. Range is the minimum and maximum.

Operation	Median (s)	Range (s)
Three-candidate encryption	0.152	0.151–0.152
Bit proofs, including opening checks	1.088	1.086–1.127
Full ballot, including the sum proof	1.137	1.135–1.210
Public verification of one ballot	0.827	0.824–0.833

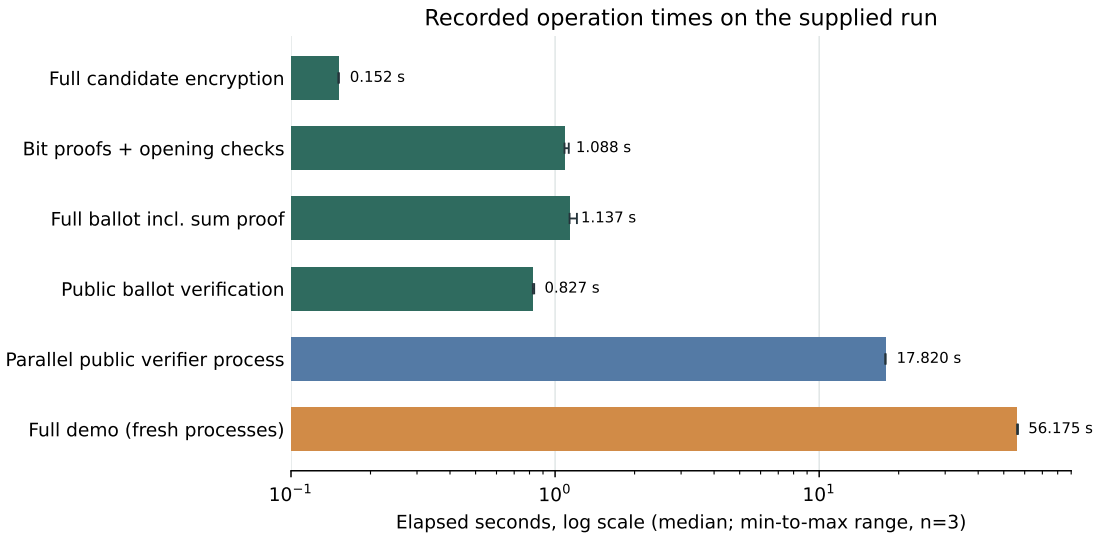


Figure 2: Medians and minimum-to-maximum ranges from the supplied measurement record, three samples each, rounded to three decimal places. The axis is logarithmic elapsed time.

partial decryptions. Each pure operation has three samples. Three fresh demonstration-process and node runs took 56.591 s, 56.175 s, and 56.165 s. Each sample is a fresh demonstration process and node; cache state was not controlled. Three isolated verifier processes took 17.820 s, 17.830 s, and 17.815 s.

Bit-proof time includes ciphertext setup and opening checks. Full-ballot time includes those proofs and the sum proof; it does not isolate the sum proof. Figure 2 shows the same three-sample medians and ranges. Three samples support a median and a range. They do not support a confidence interval or a prediction for another processor.

The two payloads together contain 76,633 bytes, including headers and the spoiled ballot. Proof bytes in those batches total 50,974. One canonical cast record in the measurement file is 10,622 bytes. Each of the three partial decryptions used 219,600 gas. These are recorded baseline values, not new benchmark results for revised code.

5.1 Scope of the measurements

These measurements cover one small demonstration on one local Anvil process: six cast ballots, one spoiled ballot, three candidates, two machines, two batches, and three partial decryptions. The implementation's current limits are 16 machines, a tally bound of 16,000, and a 16 MiB payload. No large-batch, throughput, national-scale, restart-recovery, or accessibility experiment was performed. Section 4.2 scripts ten faults; it is not a measured outage rate. No linear extrapolation from these observations is warranted.

Table 4: Payload size and gas for the two published batches on the local chain. Gas is an execution unit, not a currency fee.

Batch	Payload (bytes)	Gas	Cast ballots
First machine, mixed	44,416	1,802,170	3
Second machine, cast only	32,217	1,314,280	3

6. Related work

The 2008 Helios system described by Adida uses encrypted ballots and a mixnet; its remote-voting setting has a different trust boundary from a polling-place machine and does not remove coercion [1]. Benaloh's verifiable elections are the reference for a voter checking an encrypted choice [2]. ElectionGuard specifies a full system, with a different group and encryption construction; nothing here claims compatibility or equal assurance [3]. Scantegrity II puts end-to-end checks on optical-scan paper [4]. vVote describes a polling-place system whose operational assumptions are not the ones implemented here [6].

Those systems take on credentials, paper, or a mixnet. This one takes on two narrower jobs: the announced station total, and the person or the card that carries the only copy.

Park, Specter, Narula, and Rivest argue that moving an election onto a network or a chain can add malware, coercion, censorship, and a weak dispute story rather than remove them [9]. Table 2 is the narrower claim. With the same two slots and the same local check, a policy includes the batch only when it can read a copy that still validates and a slot can reach an open board. An altered copy that was never sent does not block a later intact copy. A mined rejection does not itself consume the machine's publication. Risk-limiting audits need voter-verifiable paper records [8]. A ciphertext receipt does not replace a ballot the voter marked.

7. Limits

The dealer is a single trusted setup. There is no distributed key generation. Three colluding trustees can read individual ballots, and the demonstration holds every share. The implementation is ordinary Python and does not protect secrets from a program that logs them. The simulated seal, power cut, reinstall, and hash comparison do not prove that a particular device ran a particular binary.

There is no voter list, eligibility check, turnout reconciliation, or one-person-one-vote check. The contract does not verify ballot proofs, trustee proofs, or tally arithmetic. Three colluding trustees can establish an incorrect pin; insufficient valid contributions stop the audit. The local chain can be reset, and an authentic complete export remains an external assumption. The comparison in Section 4.2 is ten scripted faults on one signed batch, plus one recovery run that mines reverting transactions. Each policy is reverted to the same unpublished snapshot. A down link and an unreachable board are attempts that are not transmitted. The Anvil receipt did not separate inclusion gas from revert gas. The comparison does not estimate how often a file is withheld or lost. It does not cover power loss, restart recovery, inaccessible verification tools, or human challenge participation. The other revised tests cover failed optional carrier writes and conflicting trustee submissions. A missing complete batch is rejected, not recovered. Detection provides neither automatic annulment nor a dispute remedy.

8. Conclusion

The same signed bytes verify the same way whichever source supplies them. What changes is which source a policy is willing to read. Either includes the batch when at least one copy still validates, the key is present, and one of the two slots can reach an open board. Memory includes it only from memory, and file only from the file. A missing key, a deadline that closes before a slot can send, and a board that stays unreachable stop all three. A mined revert does not mark the machine published. The intact post is still accepted while the deadline is open, and a second post is not. The receipt gas of the included transaction was 339,720, and the mined revert receipts returned the same number, so no revert-cost difference is claimed. The tally is still recomputed from the published ballots, so the machine does not announce the station total. A missing batch is a registered machine absent from that set. A hand-marked paper ballot remains the record an audit can correct by reading what the voter marked.

Availability

The revised executable checks are identified by source revision `8df8e3a`; the historical timing and gas record is retained from revision `f8d9358`. The recorded artifacts are `demo/results/measurements.json`, `demo/results/evidence.json`, `demo/results/manifest.json`, `demo/results/receipts.json`, and `demo/results/verifier-report.json`. The `Makefile` defines `make test` and `make attacks`. The revised regression sources are `tests/test_machine.py`, `tests/test_transport.py`, `tests/test_chain.py`, `tests/test_verifier.py`, and `tests/test_demo.py`; the publication-path comparison is `urna/publication.py` and `tests/test_publication.py`; the intent-comparison examples are in `tests/test_attacks.py`. These measurements describe the baseline and are not automatically results for revised code. The gas in Section 4.2 is from this policy comparison on a local Anvil 1.8.5 chain, not from that historical record. The code is under the Apache License 2.0. The public source is https://github.com/Hmenez3s/urna-aberta. No DOI or arXiv identifier exists yet; no journal publication is claimed.

References

[1] Ben Adida. Helios: Web-based open-audit voting. In *17th USENIX Security Symposium*, 2008.

[2] Josh Benaloh. Simple verifiable elections. In *Proceedings of the USENIX/Accurate Electronic Voting Technology Workshop*, 2006.

[3] Josh Benaloh, Michael Naehrig, and Olivier Pereira. Electionguard design specification. Technical specification Version 2.1.0, Microsoft Research, August 2024.

[4] David Chaum, Richard Carback, Jeremy Clark, Aleksander Essex, Stefan Popoveniuc, Ronald L. Rivest, Peter Y. A. Ryan, Emily Shen, and Alan T. Sherman. Scantegrity ii: End-to-end verifiability for optical scan election systems using invisible ink confirmation codes. In *Proceedings of the USENIX/ACCURATE Electronic Voting Technology Workshop*, 2008.

[5] David Chaum and Torben Pryds Pedersen. Wallet databases with observers. In *Advances in Cryptology, CRYPTO '92*, volume 740 of *Lecture Notes in Computer Science*. Springer, 1993.

[6] Chris Culnane, Peter Y. A. Ryan, Steve Schneider, and Vanessa Teague. vvote: a verifiable voting system. arXiv:1404.6822v4, 2015.

[7] Tero Kivinen and M. J. Kojo. More modular exponential (modp) diffie-hellman groups for internet key exchange (ike). RFC 3526, 2003.

[8] Mark Lindeman and Philip B. Stark. A gentle introduction to risk-limiting audits. *IEEE Security and Privacy*, 10(5), 2012.

[9] Sunoo Park, Michael Specter, Neha Narula, and Ronald L. Rivest. Going from bad to worse: from internet voting to blockchain voting. *Journal of Cybersecurity*, 7(1), 2021.

[10] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11), 1979.

[11] Tribunal Superior Eleitoral. Oportunidades de auditoria e fiscalização. https://www.justicaeleitoral.jus.br/urna-eletronica/oportunidades-de-auditoria-e-fiscalizacao.html, 2026.

[12] Tribunal Superior Eleitoral. Resolução no 23.751, de 26 de fevereiro de 2026. https://www.tse.jus.br/legislacao/compilada/res/2026/resolucao-no-23-751-de-26-de-fevereiro-de-2026, 2026. Arts. 231, 233, 235, 237 to 239, and 272.